Algorithmic Trading Backtester

Project Documentation: Algorithmic Trading Backtester

Version 1.0

Author: Antonio Machuca

Date: June 8, 2026

Contents

1 Introduction

1.1 Motivation

Describe the motivation behind the creation of a high-performance, vectorized algorithmic trading backtesting system.

1.2 Purpose and Objectives

Define the purpose of the document. Highlight the goal of achieving an asymptotic time complexity close to $\mathcal{O}(1)$ for financial array operations.

1.3 Scope

Define the scope of the software. Detail the strict separation of concerns between core logic (`src/core/`) and visual analysis (`notebooks/`).

1.4 Acronyms and Abbreviations

- **API**: Application Programming Interface.
- **SMA**: Simple Moving Average.
- **GBM**: Geometric Brownian Motion.
- **MDD**: Maximum Drawdown.
- **CAGR**: Compound Annual Growth Rate.

1.5 References

- Yves Hilpisch, *Python for Finance: Mastering Data-Driven Finance (2nd Edition)*.

2 Theoretical Foundations

2.1 Financial Time Series Modeling

Provide the formal mathematical definition of Geometric Brownian Motion (GBM) used for mock data generation.

2.2 Trading Strategies

2.2.1 Simple Moving Average (SMA)

Define the formal mathematical model for the SMA strategy. Example: $SMA_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$.

2.2.2 Momentum

Define the log-returns calculation: $r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$.

2.3 Performance Metrics

Provide the strict formal formulation for risk and performance metrics:

- **Sharpe Ratio**: $S = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p}$
- **Maximum Drawdown (MDD)**: $MDD = \min\left(\frac{P_t - \text{Peak}_t}{\text{Peak}_t}\right)$
- **Compound Annual Growth Rate (CAGR)**: $CAGR = \left(\frac{EV}{BV}\right)^{\frac{1}{n}} - 1$

3 Architecture and Structural Design

3.1 System Architecture

Description of the Hexagonal Architecture pattern (Ports and Adapters) and SOLID principles (ISP and DIP) applied to decouple the data ingestion from the core logic.

3.2 UML Class Design

Detailed structural explanation of the core domain classes:

- `DataHandler` (Interface and Adapters)
- `Strategy` (Strategy Pattern)
- `Portfolio` (Aggregate Root)
- `BacktestEngine` (Orchestrator)

3.3 Diagrams

(Space to include the UML Class Diagram `ClassDiagram.svg` generated in the `docs/architecture/` directory).

4 Implementation and Vectorization

4.1 Vectorization over Iteration

Explain the strict paradigm of avoiding `for` or `while` loops. Detail how `pandas` and `NumPy` were used to achieve $\mathcal{O}(1)$ asymptotic complexity for array operations across time-series data.

4.2 Structured Programming

Detail the adherence to academic structured programming paradigms, including the absolute prohibition of control flow interruption statements like `break` or `continue`.

4.3 Friction Models

Describe the implementation of the `IFrictionModel` for transaction costs calculation without loops, strictly using dataframe shift operations.

5 Results and Conclusions

5.1 Backtest Execution Analysis

Analysis of the performance and execution time. Showcase the output of the Jupyter notebook visualizations (`matplotlib` and `plotly`) plotting the *Equity Curve*.

5.2 Conclusions

Summary of the project's achievements against its initial goals in terms of software architecture, algorithmic efficiency, and mathematical rigor.

5.3 Future Work

Potential improvements, such as adding new statistical models, expanding the framework to multi-asset portfolios, or integrating real-time live trading connectivity.